

Egesha Smart Parking System

Web-based parking automation for Kenya (Python, Flask, SQLite). *Egesha* is Swahili for "park".

Run

```
pip install flask
python app.py          # open
http://127.0.0.1:5000
```

Push to GitHub: `git init && git add . && git commit -m "Egesha parking" && git remote add origin <url> && git push -u origin main`

1. Requirements analysis → modules

Requirement in the brief	Module	Code
Drivers see free slots before entry	Availability Display	<code>/api/slots</code> , bay grid
Record vehicle on	Entry and Slot	<code>/api/entry</code>

Requirement in the brief	Module	Code
arrival	Manager	
Calculate time and amount on exit	Billing	<code>minutes_parked ,</code> <code>compute_fee</code>
Barrier opens after payment	Payment and Barrier	<code>/api/pay ,</code> <code>release</code>
Keep records, revenue	Database and Reports	SQLite, <code>/api/report</code>

Use cases: driver views availability; driver enters; driver requests bill; driver pays and exits; manager views revenue.

2. Algorithms (a)

Entry

1. Clean plate (uppercase, no spaces) and validate. Reject if invalid.
2. If plate is in `active` , reject (already parked). If `free_heap` is empty, reject (full).

3. `slot = heappop(free_heap)` (lowest-numbered free bay).
4. Insert ticket (plate, slot, entry time); mark slot occupied; `active[plate] = ticket`.
5. Open barrier, show slot to driver.

Billing and exit

1. Look up `active[plate]`. If missing, reject.
2. `minutes = ceil((now - entry_time) / 60)`.
3. Scan tariff tiers in ascending order; fee is the first tier where `minutes <= max_minutes` (last tier has no limit).
4. If fee = 0, go to step 6. Otherwise show the bill and wait for payment.
5. On payment success (M-Pesa number validated, reference stored), record payment.
6. Release: set exit time, duration, fee, paid; mark slot free; `heappush(free_heap, slot)`; remove from `active`; open barrier.

Availability display: read all slots; a slot is occupied if its id is in the set of `active` slot ids; return counts.

The page polls every 5 seconds.

Tariff: ≤ 30 min free, ≤ 2 h Kshs. 50, ≤ 4 h Kshs. 100, ≤ 6 h Kshs. 300, over 6 h Kshs. 500.

3. Data structures and why (b)

Structure	Used for	Reason
Min-heap free_heap	Free slot numbers	Allocation and release in $O(\log n)$; always gives the bay nearest the entrance
Hash map active	Plate → ticket	$O(1)$ duplicate check on entry and lookup on exit; no scan of all vehicles
Ordered list of tiers	Fee rules	Tiny ($k=5$), ordered scan is simple and $O(k)$; stored in DB so management can edit it
Array/list of slots	Display grid	Fixed order, direct rendering
SQLite tables	Permanent records	Survive restarts; heap and map are rebuilt from them on start-up

A lock serialises entry, billing and payment so two cars can never get the same bay.

4. Dynamic database (c)

"Dynamic" here means slots and tariffs are rows, not code: add bays or change fees with SQL and no redeploy, and the heap and display adapt.

```
slots(slot_id PK, zone, status)
1 —< tickets
tariffs(id PK, max_minutes NULL=∞, fee)
tickets(ticket_id PK, plate, slot_id FK,
entry_time, exit_time, duration_min, fee,
paid) 1 —< payments
payments(payment_id PK, ticket_id FK,
amount, method, reference, paid_at)
```

Example: `UPDATE tariffs SET fee=60 WHERE max_minutes=120;` or `INSERT INTO slots(slot_id, zone) VALUES(41, 'C');` then restart.

Limitations and next steps

- M-Pesa is simulated. Replace the reference in `/api/pay` with a Safaricom Daraja STK push and callback.
- The barrier is a screen animation; real hardware would be driven by a relay or controller.

- No login: add staff authentication before exposing `/api/report` .